1.3 EXERCISES

1. ```
   #include <iostream>
   using namespace std;

   int main()
   {
       double hours, wage;

       cout << "Enter hours worked: ";
       cin >> hours;

       cout << "Enter wage per hour: ";
       cin >> wage;

       cout << "Pay is $" << wage * hours;

       return 0;
   }
   ```

2. ```
   #include <iostream>
   using namespace std;

   int main()
   {
       double feet;

       do
       {
           cout << "Enter feet (0 to quit): ";
           cin >> feet;

           cout << "feet * 12 << " inches\n";
       }
       while(feet != 0.0);

       return 0;
   }
   ```

3. ```
   /*
       This program computes the lowest
       common denominator.
   */
   #include <iostream>
   using namespace std;

   int main()
   {
       int a, b, d, min;

       cout << "Enter two numbers: ";
   ```
   ```

```
    cin >> a >> b;

    min = a > b ? b : a;

    for(d = 2; d<min; d++)
        if(((a%d)==0) && ((b%d)==0))
            break;
    if(d==min)
    {
        cout << "No common denominators\n";
        return 0;
    }
    cout << "The lowest common denominator is " << d << " \n"
       ;

    return 0;
}
```

1.4 EXERCISES

1. The comment, although strange, is valid.

1.5 EXERCISES

2.
```
#include <iostream>
#include <cstring>
using namespace std;

class card
{
    char title[80]; // book title
    char author[40]; // author
    int number; // number in library
public:
    void store(char *t, char *name, int num);
    void show();
};

void card::store(char *t, char *name, int num)
{
    strcpy(title, t);
    strcpy(author, name);
    number = num;
}

void card::show()
{
    cout << "Title: " << title << "\n";
    cout << "Author: " << author << "\n";
    cout << "Number on hand: " << number << "\n";
```

```
}

int main()
{
    card book1, book2, book3;

    book1.store("Dune", "Frank Herbert", 2);
    book2.store("The Foundation Trilogy", "Isaac Asimov", 2);
    book3.store("The Rainbow", "D. H. Lawerence", 1);

    book1.show();
    book2.show();
    book3.show();

    return 0;
}
```

3.
```
#include <iostream>
using namespace std;

#define SIZE 100

class q_type
{
    int queue[SIZE]; // holds the queue
    int head, tail; // indices of head and tail
public:
    void init(); // initialize
    void q(int num); // store
    int deq(); // retrieve
};

// Initialize
void q_type::init()
{
    head = tail = 0;
}

// Put value on the queue.
void q_type::q(int num)
{
    if(tail+1==head || (tail+1==SIZE && !head))
    {
        cout << "Queue is full\n";
        return;
    }
    tail++;
    if(tail==SIZE)
        tail = 0; // cycle around
    queue[tail] = num;
}
```

```
// Remove a value from a queue.
int q_type::deq()
{
    if(head == tail)
    {
        cout << "Queue is empty\n";
        return 0; // or some other error indicator
    }
    head++;
    if(head==SIZE)
        head = 0; // cycle around
    return queue[head];
}

int main()
{
    q_type q1, q2;
    int i;

    q1.init();
    q2.init();

    for(i=1; i<=10; i++)
    {
        q1.q(i);
        q2.q(i*i);
    }

    for(i=1; i<=10; i++)
    {
        cout << "Dequeue 1: " << q1.deq() << "\n";
        cout << "Dequeue 2: " << q2.deq() << "\n";
    }

    return 0;
}
```

1.6 EXERCISES

1. The function **f()** is not prototyped.

1.7 EXERCISES

1. ```
   #include <iostream>
   #include <cmath>
   using namespace std;

   // Overload sroot() for integers, longs, and doubles.

   int sroot(int i);
   ```
   ```

```
long sroot(long i);
double sroot(double i);

int main()
{
    cout << "Square root of 90.34 is : " << sroot(90.34);
    cout << "\n";
    cout << "Square root of 90L is : " << sroot(90L);
    cout << "\n";
    cout << "Square root of 90 is : " << sroot(90);
    cout << "\n";

    return 0;
}

// Return square root of integer.
int sroot(int i)
{
    cout << "computing integer root\n";
    return (int) sqrt((double) i);
}

// Return square root of long.
long sroot(long i)
{
    cout << "computing long root\n";
    return (long) sqrt((double) i);
}

// Return square root of double.
double sroot(double i)
{
    cout << "computing double root\n";
    return sqrt(i);
}
```

2. The functions **atof()**, **atoi()**, **atol()** cannot be overloaded because they differ only in the type of data they return. Function overloading requires that either the type or the number of arguments differs.

3. ```
   // Overload the min() function.

   #include <iostream>
   #include <cctype>
   using namespace std;

   char min(char a, char b);
   int min(int a, int b);
   double min(double a, double b);

   int main()
   {
   ```
   ```

```
    cout << "Min is: " << min('x', 'a') << "\n";
    cout << "Min is: " << min(10, 20) << "\n";
    cout << "Min is: " << min(0.2234, 99.2) << "\n";

    return 0;
}

// min() for chars
char min(char a, char b)
{
    return tolower(a)<tolower(b) ? a : b;
}

// min() for ints
int min(int a, int b)
{
    return a<b ? a : b;
}

// min() for doubles
double min(double a, double b)
{
    return a<b ? a : b;
}
```

4.
```
#include <iostream>
#include <cstdlib>
using namespace std;

// Overload sleep to accept integer or char * argument
void sleep(int n);
void sleep(char *n);

// Change this value to fit your processor speed.
#define DELAY 100000

int main()
{
    cout << ".";
    sleep(3);
    cout << ".";
    sleep("2");
    cout << ".";

    return 0;
}

// Sleep() with integer argument.
void sleep(int n)
{
    long i;
```

```
    for( ; n; n--)
        for(i=0; i<DELAY; i++);
}

// Sleep() with char * argument.
void sleep(char *n)
{
    long i;
    int j;

    j = atoi(n);

    for( ; j; j--)
        for(i=0; i<DELAY; i++);
}
```

MASTERY SKILLS CHECK: Chapter 1

1. Polymorphism is the mechanism by which one general interface can be used to access many specific implementations. Encapsulation provides a protected linkage between code and its related data. Access to encapsulated routines can be tightly controlled, thus preventing unwanted tampering. Inheritance is the process by which one object can acquire the traits of another. Inheritance is used to support a system of hierarchical classification.

2. You can include comments in a C++ by using either the normal C-style comments or the C++-specific single-line comments.

3. ```
   #include <iostream>
   using namespace std;

   int main()
   {
       int b, e, r;
       cout << "Enter base: ";
       cin >> b;
       cout << "Enter exponent: ";
       cin >> e;

       r = 1;
       for( ; e; e--)
           r = r * b;

       cout << "Result: " << r;

       return 0;
   }
   ```

4. ```
   #include <iostream>
   #include <cstring>
   using namespace std;
   ```

```
// Overload string reversal function.
void rev_str(char *s); // reverse string in place
void rev_str(char *in, char *out); // put reversal into out

int main()
{
    char s1[80], s2[80];

    strcpy(s1, "This is a test");

    rev_str(s1, s2);
    cout << s2 << "\n";

    rev_str(s1);
    cout << s1 << "\n";

    return 0;
}

// Reverse string, put result in s.
void rev_str(char *s)
{
    char temp[80];
    int i, j;

    for(i=strlen(s)-1, j=0; i>=0; i--, j++)
        temp[j] = s[i];

    temp[j] = '\0'; // null-terminate result

    strcpy(s, temp);
}

// Reverse string, put result into out.
void rev_str(char *in, char *out)
{
    int i, j;

    for(i=strlen(in)-1, j=0; i>=0; i--, j++)
        out[j] = in[i];

    out[j] = '\0'; // null-terminate result
}
```

5.
```
#include <iostream.h>

int f(int a);

int main()
{
    cout << f(10);
```

```
    return 0;
}

int f(int a)
{
    return a * 3.1416;
}
```

6. The **bool** data type stores Boolean values. The only values an object of type **bool** can have are **true** and **false**.

REVIEW SKILLS CHECK: Chapter 2

1.
```
#include <iostream>
#include <cstring>
using namespace std;

int main()
{
    char s[80];

    cout << "Enter a string: ";
    cin >> s;

    cout << "Length: " << strlen(s) << "\n";

    return 0;
}
```

2.
```
#include <iostream>
#include <cstring>
using namespace std;

class addr
{
    char name[40];
    char street[40];
    char city[30];
    char state[3];
    char zip[10];
public:
    void store(char *n, char *s, char *c, char *t, char *z);
    void display();
};

void addr::store(char *n, char *s, char *c, char *t, char *z)
{
    strcpy(name, n);
    strcpy(street, s);
    strcpy(city, c);
    strcpy(state, t);
```